

# Mini-proiecte — multithreading

Lotul 1 · repo `multithreading-projects` · Java · Alex

**Cum se lucrează aici.** Nu primești pași și nu primești hinturi. Primești o cerință și un criteriu de acceptare. Clasele și le proiectezi singur.

1. Cerința **nu-ți spune ce clasă din Java să folosești** — alegerea unelei face parte din proiect. Dacă nu știi care e, caută în `multithreading/src/teorie/` după *comportamentul* de care ai nevoie, nu după nume.
2. Fiecare program **își tipărește singur verdictul** la final: așteptat vs. real, sau o durată comparată cu alta. Fără verdict, proiectul nu e terminat.
3. Fiecare program **se închide singur**, fără `Ctrl+C`.
4. Un mini-proiect = un folder = o clasă `MNN`.

## M01 — Descărcător de pagini

### Cinci pagini, în același timp

ușor

Programul „descarcă” 5 pagini web. Nu descarci nimic real: fiecare pagină are un nume și o durată între 200 și 1200 ms, pe care o simulezi făcând programul să aștepte acel timp.

Pornești toate descărcările, le lași să se desfășoare simultan, și abia după ce **toate** s-au terminat afișezi un raport cu: fiecare pagină și durata ei, care a durat cel mai mult, durata totală a programului.

**Criteriu de acceptare:** durata totală afișată e apropiată de durata celei mai lente pagini, nu de suma tuturor. Programul afișează ambele numere și scrie singur verdictul: „în paralel” dacă totalul e sub jumătate din sumă, „secvențial” altfel. Raportul apare o singură dată, după ce toate cele 5 s-au terminat.

## M02 — Cronometrul păcălit

### Aceeși muncă, două durate

ușor

Ai 4 sarcini identice, fiecare durează 500 ms. Le rulezi în două variante și cronometrezi fiecare variantă:

- **Varianta A** — sarcinile chiar se desfășoară în același timp.
- **Varianta B** — codul arată aproape la fel, dar tot ce se întâmplă se întâmplă pe firul principal, unul după altul.

Diferența dintre A și B trebuie să fie de **un singur cuvânt** în cod. La final afișezi, pentru fiecare variantă, durata măsurată și numele firelor care au lucrat efectiv.

**Criteriu de acceptare:** varianta A  $\approx$  500 ms, cu nume de fire diferite între ele. Varianta B  $\approx$  2000 ms, cu toate sarcinile raportând același nume de fir. Programul tipărește la final „A a fost de N ori mai rapid”, cu N calculat.

## M03 — Jurnal de fundal

### Cine ține programul în viață

mediu

Aplicația procesează 20 de comenzi, una după alta, fiecare durând între 50 și 150 ms. În paralel rulează un jurnal: la fiecare 100 ms scrie o linie cu câte comenzi s-au procesat până atunci.

Jurnalul **nu știe** dinainte cât durează prelucrarea și nu are un număr fix de linii de scris — scrie la nesfârșit, cât timp există aplicație. Când cele 20 de comenzi s-au terminat, programul afișează un sumar și se închide imediat.

**Criteriu de acceptare:** programul se termină singur, fără `Ctrl+C`, în aproximativ timpul celor 20 de comenzi. **Ultima** linie din output e sumarul, nu o linie de jurnal. În timpul rulării au apărut linii de jurnal intercalate cu comenzile.

## M04 — Calculator de facturi

### Rezultate cerute în altă ordine decât sosesc

mai greu

Ai 6 facturi, fiecare cu un număr și o listă de sume. Pentru fiecare se calculează totalul, iar calculul durează — simulezi cu o așteptare: factura 0 durează cel mai mult, factura 5 cel mai puțin.

Factura 3 conține o sumă invalidă. Calculul ei trebuie să arunce o excepție cu mesajul `Invalid amount on invoice 3`.

Calcululele se desfășoară în același timp, dar **rezultatele le ceri în ordinea facturilor**: întâi 0, apoi 1, și așa mai departe. Pentru fiecare afișezi, în ordine, o linie cu: numărul facturii, totalul (sau motivul erorii) și la câte milisecunde de la pornirea programului ai obținut acel rezultat.

**Criteriu de acceptare:** pentru factura 3 afișezi `Invalid amount on invoice 3`, nu numele unei excepții de învelis. Prima linie apare târziu, iar următoarele apar aproape instantaneu una după alta — și **explici în output, într-o singură propoziție scrisă de tine, de ce se întâmplă asta**. Celelalte 5 facturi au totalul corect.

## M05 — Contor de vizite

### Cât protejezi, exact

mai greu · 3 stagii

Un site numără vizitele. 4 fire de execuție înregistrează fiecare 50.000 de vizite, deci la final contorul trebuie să arate exact 200.000.

Fiecare înregistrare face **două** lucruri: crește contorul, apoi pregătește un raport — o operație lentă care **nu** atinge contorul (simulezi cu un mic calcul care consumă timp).

Scrii trei stagii, în aceeași clasă, și le rulezi pe rând:

- **Stagiul A** — varianta naivă. Contorul iese greșit.
- **Stagiul B** — repari, protejând toată înregistrarea de vizită.
- **Stagiul C** — repari, protejând doar partea care are nevoie.

Fiecare stagiou afișează valoarea obținută, valoarea așteptată și durata.

**Criteriu de acceptare:** stagiul A afișează un număr mai mic decât 200.000 și cât s-a pierdut. Stagiile B și C afișează exact 200.000. Stagiul C e vizibil mai rapid decât B, iar programul tipărește raportul dintre cele două durate.

